

Lösungsblatt: Systemarchitekturen zur Softwareentwicklung

Modul 321 – Lektion 3

Hinweis: Dieses Lösungsblatt enthält Musterlösungen und einen möglichen Erwartungshorizont. Andere Antworten sind korrekt, wenn sie fachlich sauber begründet sind und die Architekturmerkmale nachvollziehbar eingeordnet werden.

1. Begriffe erklären

Begriff	Mögliche Lösung
Monolith	Ein Monolith ist eine Anwendung, die als zusammenhängendes System entwickelt und meist gemeinsam deployt wird. Viele fachliche Teile liegen in derselben Codebasis und laufen im selben Hauptsystem.
Microservices	Microservices teilen ein System in mehrere eigenständige Dienste auf, die über APIs oder Messaging miteinander kommunizieren. Jeder Dienst hat idealerweise eine klar abgegrenzte fachliche Verantwortung.
Client-Server	Beim Client-Server-Modell sendet ein Client Anfragen an einen Server, der Daten oder Funktionen bereitstellt. Typische Clients sind Browser oder Apps, typische Server sind Web- oder API-Dienste.
Peer-to-Peer	Beim Peer-to-Peer-Modell kommunizieren Teilnehmer direkt miteinander und übernehmen ähnliche Rollen im Netzwerk. Es gibt nicht zwingend einen zentralen Hauptserver für alle Datenflüsse.

2. Beispiele zuordnen

1. Monolith
2. Client-Server
3. Microservices
4. Peer-to-Peer
5. Monolith
6. Peer-to-Peer

Mögliche Begründungen:

- Beispiel 1 ist ein Monolith, weil die Anwendung als gemeinsames System mit zentraler Datenbank betrieben wird.
- Beispiel 2 ist Client-Server, weil der Browser als Client eine zentrale API als Server anspricht.
- Beispiel 3 ist ein Microservice-System, weil mehrere fachlich getrennte Dienste über Schnittstellen zusammenarbeiten.

3. Monolith oder Microservices?

1. Als Startpunkt ist ein gut strukturierter Monolith naheliegend.
2. Gründe: kleines Team, geringe Betriebs-Komplexität, schnelle Änderungen, enger fachlicher Zusammenhang und noch wenig Last.
3. Ein mögliches Risiko ist, dass bei starkem Wachstum die Kopplung zunimmt und unabhängige Releases oder gezielte Skalierung schwieriger werden.
4. Wichtig sind saubere Modulgrenzen, gute Tests, klare Verantwortlichkeiten im Code und bewusstes Vermeiden unnötiger Vermischung von Fachlichkeiten.

4. Microservices kritisch beurteilen

1. Vorteile: unabhängige Deployments, gezielte Skalierung, fachlich klarere Teamverantwortung.
2. Herausforderungen: Netzwerkfehler und Timeouts, verteiltes Monitoring/Tracing, schwierigere Datenkonsistenz, mehr Deployment- und Infrastrukturaufwand.
3. Weil fachliche Unklarheit oder schlechte Zusammenarbeit durch technische Aufteilung nicht verschwinden, sondern oft sichtbarer und teurer werden.
4. Sie wären eher übertrieben bei kleinen Teams, einfachen Produkten oder frühen Projektphasen mit häufigen Richtungswechseln.

5. Client-Server vs. Peer-to-Peer

Frage	Client-Server	Peer-to-Peer
Wer stellt typischerweise die zentrale Logik bereit?	Der Server	Die Logik ist auf mehrere Teilnehmer verteilt
Wie erfolgt die Datenkontrolle oder Datenspeicherung?	Oft zentral oder serverseitig kontrolliert	Häufig verteilt auf mehrere Knoten
Wo ist der Betrieb meist einfacher?	Im Client-Server-Modell	Peer-to-Peer ist meist schwerer zu steuern und zu koordinieren
Nennen Sie ein typisches Praxisbeispiel.	Webshop, Web-API, Mobile Backend	File-Sharing, Blockchain-Netzwerk

6. Architektur eines Online-Shops

1. Als Monolith könnten Produktanzeige, Warenkorb, Checkout, Suche und Benachrichtigungen in einer gemeinsamen Anwendung mit zentraler Datenbank umgesetzt werden.
2. Als Microservice-System könnte es separate Dienste für Katalog, Warenkorb, Checkout, Suche und Benachrichtigungen geben, die über APIs oder Events kommunizieren.
3. Für ein kleines Team zu Beginn ist meist der Monolith sinnvoller, weil Entwicklung, Testing und Betrieb einfacher bleiben.
4. Bei starkem Wachstum könnten Microservices interessanter werden, wenn einzelne Teile sehr unterschiedlich skalieren oder mehrere Teams unabhängig liefern müssen.

7. Kombinationsverständnis

1. Richtig. Ein Monolith kann als zentrale Server-Anwendung betrieben werden, die von Browsern oder Apps angesprochen wird.
2. Falsch. Microservices beschreiben die Aufteilung in Dienste, nicht automatisch ein Peer-to-Peer-Kommunikationsmodell.
3. Falsch. Auch P2P-Systeme können zentrale Hilfsdienste besitzen, etwa für Registrierung, Discovery oder Authentisierung.
4. Richtig. Wenn er sauber modularisiert ist, kann ein Monolith lange tragfähig und wirtschaftlich sein.

8. Entwurfsaufgabe

Mögliche Zielarchitektur:

- Frontend im Browser
- Backend / API für Aufgaben, Material und Nachrichten
- Datenbank
- Externer Videokonferenz-Dienst
- Optional später separater Messaging- oder Datei-Service

Mögliche Antworten:

1. Eine einfache Zielarchitektur kann aus Browser-Frontend, zentralem Backend, Datenbank und angebundenem Video-Dienst bestehen.
2. Für den Start ist ein modularer Monolith meist sinnvoll, weil die Fachlichkeiten eng zusammenhängen und die Komplexität sonst unnötig steigt.
3. Das Client-Server-Muster erscheint zwischen Browser/App als Client und Backend/API als Server.
4. Später könnte zum Beispiel Videofunktion, Suche oder Nachrichtenfunktion in einen eigenen Dienst ausgelagert werden.
5. Peer-to-Peer wäre hier eher unpassend, weil die Schule meist zentrale Kontrolle, Datenschutz, einfache Administration und klare Verantwortlichkeiten braucht.

Didaktischer Hinweis

Die Lernenden sollen nicht nur Begriffe wiedergeben, sondern Architektur als Abwägung verstehen: einfache Systeme profitieren oft von einem modularen Monolithen, während Microservices erst bei klaren fachlichen Grenzen, höherer Last oder mehreren autonomen Teams ihren Aufwand rechtfertigen.